TABLE OF CONTENTS

1. Introduction 1
2. Background and Motivation 2
3. Problem Statement and Objectives 2
4. Scope and Significance 3
5. Fundamentals of Security Integrations and Data Integrity 3
6. Technology Stack 4
7. Connector Configuration, Provider Coverage and Common Data Model 4
8. System Architecture and Data Flow 6
9. Configuration-Driven Connector Runtime and Execution 7
10. Security Architecture and Credential Protection 8
11. Staged Ingestion and Integrity Evaluation 9
12. Sync Integrity Receipts and Auditability 10
13. Methodology and Processing Workflow 11
14. Incident Detection and Response 12
15. Evaluation and Test Results 13
16. Applications, Current Limitations and Future Enhancements 14
17. Conclusion and References 15

List of figures

Figure 8.1 Implemented architecture and data flow 6
Figure 11.1 Integrity decision flow 9
Figure 13.1 End-to-end processing workflow 11

Page numbers refer to the main report, whose numbering begins at the Introduction.

1. INTRODUCTION

1.1 Context

Security teams rely on many separate platforms: identity providers such as Okta, code-hosting services such as GitHub, vulnerability feeds such as the National Vulnerability Database (NVD) and the CISA Known Exploited Vulnerabilities catalogue, and detection platforms such as Elastic Security. Each platform exposes its own Application Programming Interface (API), authentication method, pagination style and data format. Bringing their data together is a prerequisite for monitoring, investigation and reporting, yet every new source traditionally requires a separately written integration.

Integrations built one at a time behave inconsistently. One connector retries a failed request while another stops; one honours a rate-limit signal while another is blocked by the provider; one quietly publishes half a dataset when a request fails part-way through. Because security decisions are made from the integrated data, these inconsistencies directly affect the quality of those decisions.

1.2 The integration-hub risk

An integration hub also concentrates credentials. To read from many platforms it must hold API keys and OAuth tokens for each of them, so a compromise of the hub can expose every connected service at once. The 2025 Salesloft Drift incident, summarised in Chapter 2, showed that this risk is practical rather than theoretical. A secure integration platform must therefore protect the credentials it holds, limit what a stolen credential can do, and detect misuse quickly.

1.3 IntegrateX

IntegrateX is a configuration-driven, provider-neutral security integration platform developed for Mitigata CyberForge, Problem Statement 05. A new data source is described in one YAML file that declares how to authenticate, which endpoints to call, how to paginate, how often to run and how to map responses into a common schema. One reusable engine executes every connector, and the engine and broker source directories contain no provider-specific code.

Beyond configuration-driven ingestion, IntegrateX contributes two architectural ideas. First, **evidence-gated synchronisation**: records are staged, evaluated for completeness and quality, and published only when the configured requirements are met, with every decision recorded in a tamper-evident **Sync Integrity Receipt**. Second, a **connector security layer** built around an isolated credential broker, complemented by sender-constrained tokens where providers support them, decoy credentials, provider-aware IP monitoring and a human-authorised kill switch.

1.4 Report organisation

Chapters 2 to 5 present the motivation, problem, scope and technical fundamentals. Chapters 6 to 12 describe the technology stack, connector model, architecture, runtime, security design, staged ingestion and receipts. Chapters 13 to 15 cover the processing workflow, incident response and evaluation. Chapter 16 discusses applications, limitations and planned work, and Chapter 17 concludes the report.

2. BACKGROUND AND MOTIVATION

Most security platforms offer Representational State Transfer (REST) APIs over HTTPS. Although the transport is common, the details differ: authentication may use a static API key, HTTP Basic credentials, a bearer token or the OAuth 2.0 client-credentials grant; results may be split into pages by a Link header, a cursor, an offset or a page number; and field names and timestamp formats vary between vendors. Writing a separate program for each combination repeats the same reliability logic many times and multiplies the places where errors can occur.

Reliability is a second concern. Network faults, server errors (HTTP 5xx) and rate limiting (HTTP 429) are normal operating conditions for external APIs. If a connector fails after reading some pages and its partial output is written straight into the shared store, downstream users cannot distinguish a complete dataset from an interrupted one. Data integrity therefore depends on knowing whether a synchronisation actually finished, not merely whether some records arrived.

The third concern is the security of the hub itself. Between March and June 2025 attackers gained access to Salesloft's GitHub environment and subsequently to the Drift application's cloud environment, where OAuth tokens held for customer integrations were stored. Between 8 and 18 August 2025 those tokens were used to export data from customers' Salesforce instances, and more than 700 organisations were reported as affected before the tokens were revoked on 20 August 2025 [15], [16]. The case illustrates three lessons used as design inputs for IntegrateX: credentials held by a hub must be isolated from the rest of the system; a stolen token should be difficult to use elsewhere; and misuse must be detected and contained quickly. IntegrateX is not related to that incident and is not claimed to have prevented it; the case is used only as motivation.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem statement

Organisations need to ingest security data from many heterogeneous APIs into one consistent store. Current approaches require repeated custom development for each provider, handle API failures inconsistently, concentrate high-value credentials in the integration hub, and may publish incomplete or malformed datasets without any verifiable record of what was fetched and why it was accepted. The problem addressed by this project is to design and implement a platform in which a new source is added through configuration alone, every connector inherits the same reliability and security controls, and no synchronised data is trusted without recorded evidence of its completeness.

3.2 Objectives

1. Onboard a provider through a validated YAML file, with no engine changes.
2. Run every connector on one runtime for authentication, paging, retries and mapping.
3. Keep credentials encrypted and accessible only to an isolated broker process.
4. Normalise records into common *users*, *devices* and *events* tables.
5. Stage each synchronisation, evaluate its integrity and publish it atomically or withhold it.
6. Record every synchronisation decision in a tamper-evident Sync Integrity Receipt.
7. Detect credential misuse and support a human-authorised incident response.

4. SCOPE AND SIGNIFICANCE

The implemented scope covers read-only ingestion from HTTPS APIs that return JSON, with five authentication types and six pagination styles. It includes the credential broker, signed configurations, scheduling, staged ingestion, integrity receipts, a hash-chained audit ledger, a web dashboard, a command-line interface (CLI), decoy credentials, foreign-IP detection from provider audit logs, a capability registry, an incident advisor and a kill switch. Writing data back to providers, GraphQL or SOAP APIs, request-signing schemes such as AWS Signature Version 4, and operating-system-level isolation of processes are outside the current scope.

The work is significant because it treats the integration layer as both a data pipeline and an attack surface. Instead of adding security controls to individual connectors, it applies one set of controls to all of them and records the evidence needed to audit each result.

5. FUNDAMENTALS OF SECURITY INTEGRATIONS AND DATA INTEGRITY

Table 5.1 summarises the concepts on which the implementation relies.

Table 5.1 Core concepts and their use in IntegrateX

Concept	**Meaning**	**Use in IntegrateX**
REST API over HTTPS	Resource-oriented web requests	All provider calls (read-only)
API key, bearer token	Static secret sent in a header	Injected only by the broker
OAuth 2.0 [6]	Service obtains a short-lived token	Token kept only in broker memory
DPoP, RFC 9449 [4]	Token bound to a client-held key	Optional; key held in broker memory
Pagination	Results split across requests	Six styles, declared per stream
Incremental sync	Fetch changes since a watermark	Advanced only after a verified run
Normalisation	Mapping fields to one schema	Users, devices and events tables
SHA-256 [11]	Cryptographic hash function	Page hashes, receipts, ledger chain
AES-256-GCM [10]	Authenticated encryption	Envelope encryption of stored secrets
HMAC, HKDF [8], [9]	Keyed integrity; key derivation	Signed configurations
Hash-chained log	Each entry covers the previous hash	Append-only audit ledger
Honeytoken	Decoy credential; use signals theft	Credential canaries
IP allow-listing	Only listed IPs may use a credential	Declared, checked and monitored

6. TECHNOLOGY STACK

IntegrateX is written in TypeScript and runs on Node.js version 22.19 or later. Table 6.1 lists the technologies present in the repository and their roles.

Table 6.1 Technology stack

Technology	Responsibility	Why it is used
TypeScript, Node.js	All services: broker, API, scheduler, worker, CLI	Static types over a single runtime for every process
Fastify	HTTP API, dashboard hosting, broker sockets	Low-overhead server with schema-friendly routing
PostgreSQL 16	Configurations, secrets, runs, staging, receipts, ledger	Transactions, row-level grants and advisory locks
pg-boss	Job queue between scheduler and worker	Durable queue stored in PostgreSQL
Zod	Connector configuration schema	Precise validation with path-specific error messages
yaml	Parsing connector files	Line numbers for validation errors
undici	Outbound HTTP inside the broker	Explicit timeouts, size limits and no automatic redirects
jose	DPoP proofs and private-key JWT assertions	Standards-based JSON Web Token and key handling
Node.js crypto	AES-256-GCM, HMAC-SHA-256, HKDF, SHA-256	Built-in, audited cryptographic primitives
pino, commander	Structured logging; the ix CLI	Machine-readable logs and a consistent command set
Anthropic SDK; Groq API	Optional incident-advisor narrative	Large language model explanation behind fixed rules
HTML, CSS, JavaScript	Web dashboard served by Fastify	No front-end framework is required
Vitest, ESLint	Automated tests and static analysis	Fast TypeScript test runner; enforced code checks

7. CONNECTOR CONFIGURATION, PROVIDER COVERAGE AND COMMON DATA MODEL

7.1 Connector structure

A connector file (API version *ix/v1*) contains a **metadata** block (identifier, source name, tenant), a **connection** block (base URL and authentication), one or more **streams** (request path and parameters, location of records in the response, pagination and field mapping), a **schedule**, a **reliability** block (timeout, retries, rate limit, circuit breaker), a **policy** block (allowed hosts, paths and methods, and per-run request and record caps), optional **credential** expectations, and an optional **capabilities** declaration. Secrets are never written in the file; it refers to them by name, for example *secret_ref: github_pat*. Values written as *${env.NAME}* are substituted from the environment before validation.

7.2 Authentication and pagination options

Five authentication types are accepted: *none*, *api_key* (header or query parameter, with an optional prefix), *basic*, *bearer* and *oauth2_client_credentials*. The OAuth type supports client-secret Basic, client-secret POST and private-key JWT client authentication [7], an optional revocation endpoint for RFC 7009 [5], and optional DPoP. Six pagination types are implemented: *link_header* (RFC 8288 [13]), *cursor*, *offset*, *page_number*, *body_link* (next URL inside the response body) and *none*. Mappings support nested paths, fallback paths, constants, concatenation, enumerated value maps, attribute selection and transforms such as ISO-8601 conversion, epoch conversion, case changes and CVSS-to-severity conversion.

7.3 Provider coverage

Table 7.1 separates connectors verified against live provider APIs during development from configurations written for vendor coverage but not yet exercised live.

Table 7.1 Connector configurations in the repository

Connector	Authentication	Pagination	Entity	Status
github-members	Bearer (PAT)	Link header	users	Live, verified
github-issues	Bearer (PAT)	Link header	events	Live, verified
github-repos	Bearer (PAT)	Link header	events	Live, verified
okta-users	OAuth client credentials	Link header	users	Live, verified
okta-logs	OAuth client credentials	Link header	events	Live, verified
nvd-cves	API key	Offset	events	Live, verified
cisa-kev	None (public feed)	None	events	Live, verified
elastic-security-alerts	API key	Offset	events	Live, verified
crowdstrike-devices	OAuth client credentials	Offset	devices	Configuration only
trendmicro-visionone-alerts	Bearer	Body link	events	Configuration only
netskope-alerts	API key	Offset	events	Configuration only
fortigate-devices	Bearer	None	devices	Configuration only

DPoP and foreign-IP detection were verified end-to-end only against the project's mock identity provider, because the live Okta connectors use standard bearer tokens. The Elastic configuration was run against a real Elastic Cloud Serverless project.

7.4 Common data model

Records are normalised into three tables. **Users** hold identifier, name, email, status and last login; **devices** hold hostname, IP address, operating system, owner and last-seen time; **events** hold type, severity, timestamp and description. Every row also stores its source, the connector that produced it, the run identifier, the time it was fetched and an *attributes* field for selected provider-specific values. The primary key combines tenant, source and source identifier, so repeated synchronisations update records instead of duplicating them.

8. SYSTEM ARCHITECTURE AND DATA FLOW

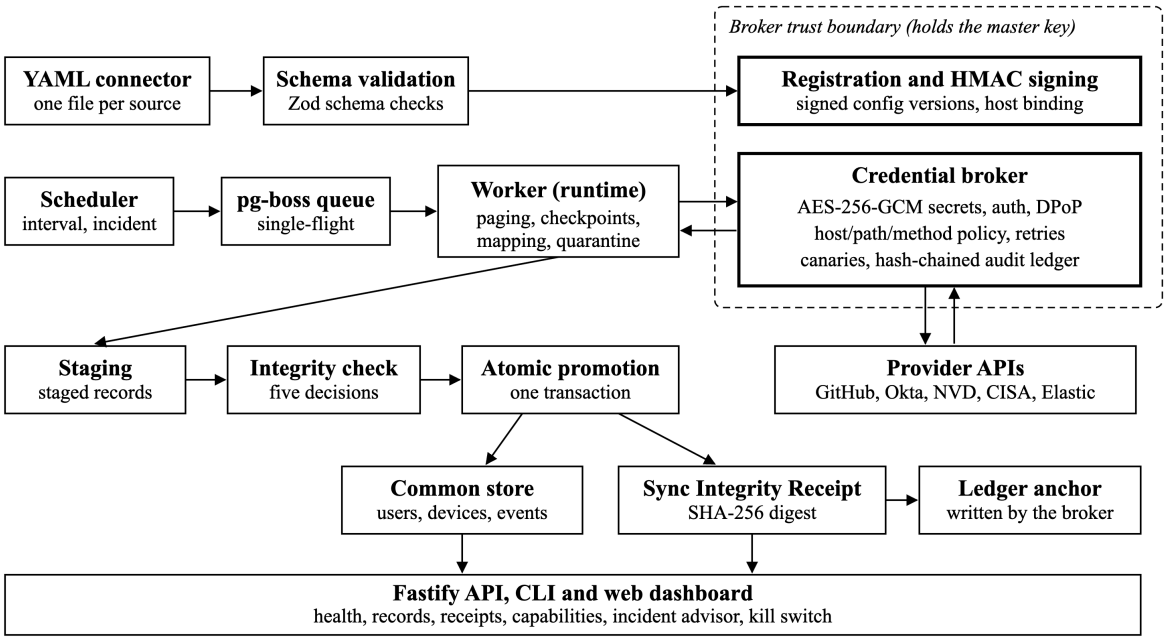


Figure 8.1 Implemented architecture and data flow

Figure 8.1 shows the components and their order of interaction. A YAML file is validated against the Zod schema and submitted to the broker, which signs the approved version with an HMAC key and records which hosts may receive each secret. The **scheduler** converts schedules into jobs on a **pg-boss** queue; the **worker** executes the connector runtime but never receives a credential. For every request it asks the **broker** to perform the call; the broker checks the signed configuration and the host, path and method policy, adds authentication, applies retries and rate limits, and appends an entry to the audit ledger.

Pages returned to the worker are paginated, checkpointed, mapped and written to **staging** tables, with invalid records placed in quarantine. When the stream ends, the **integrity evaluator** decides whether the synchronisation may be published. **Promotion** writes the run status, the new watermark, the published records and the receipt in one database transaction, after which the broker anchors the receipt digest in the ledger. The Fastify API, the CLI and the dashboard read from these tables. The broker, API, scheduler and worker run as separate processes, and the worker and scheduler refuse to start if broker or administrator secrets are present in their environment.

9. CONFIGURATION-DRIVEN CONNECTOR RUNTIME AND EXECUTION

At load time the YAML text is parsed, environment references are substituted, and the result is validated; errors are reported with the line and field path, and policy warnings are raised for risks such as a missing host allow-list. A validated file is registered through the broker, which re-validates, hashes and signs it. At run time the engine reads only signed versions.

For each stream the runtime builds the first request, sends it through the broker, and checks that the response contains the declared records location; a successful status with an unexpected body shape fails the run instead of being treated as empty. The paginator then produces the next request or declares the stream complete. Request cycles, repeated cursors and pages identical to earlier pages are treated as anomalies, because they indicate that the provider is ignoring the paging parameters.

Requests that time out or return a server error are retried with exponential backoff (by default three retries with a 1 s base and ±10 % jitter, giving delays of about 1 s, 2 s and 4 s). HTTP 429 responses are handled by honouring the Retry-After header up to a configured maximum, and a token-bucket limiter enforces the configured requests per minute. After five consecutive failed runs a circuit breaker pauses the connector for ten minutes by default. Each run holds a PostgreSQL advisory lock so that only one run per connector is active, and checkpoints allow an interrupted synchronisation to resume. Incremental streams add a cursor parameter derived from the last committed watermark, and accepted records are written with upserts. Listing 9.1 is an excerpt of a connector from the repository.

Listing 9.1 Excerpt of connectors/github-repos.yaml

```
connection:
  base_url: https://api.github.com
  auth: { type: bearer, secret_ref: github_pat }
streams:
  - name: repos
    entity: event
    request: { method: GET, path: /user/repos, params: { per_page: 100, sort: pushed } }
    records_path: $
    pagination: { type: link_header, rel: next }
    mapping:
      id:          { path: id, required: true }
      type:        { const: repository }
      severity:    { path: visibility, map_enum: { public: medium, internal: low, private: info } }
      timestamp:   { path: [pushed_at, updated_at], transform: to_iso8601 }
      description: { path: full_name, required: true }
schedule: { every: 6h, jitter: 5m }
policy: { allowed_hosts: [api.github.com], allowed_paths: [/user/repos], methods: [GET] }
```

The excerpt omits the reliability, credential and alert blocks of the full file. Adding this connector required no change to the engine.

10. SECURITY ARCHITECTURE AND CREDENTIAL PROTECTION

10.1 Broker isolation and secret storage

The credential broker is the only process that holds the master key and the database role able to read the credentials table. It listens on two Unix domain sockets with file mode 0600: a request socket used by the worker and scheduler, which can only ask for calls to be made, and an administration socket for registration, secrets, re-enabling and the kill switch, protected by a separate administrator token. Each secret is encrypted with AES-256-GCM under its own random 256-bit data key, which is in turn encrypted under the master key; the tenant and secret name are bound as additional authenticated data. Secret values are write-only: no command or API returns them.

10.2 Signed configurations and host binding

Each approved configuration version is signed with HMAC-SHA-256 using a key derived from the master key by HKDF. The broker honours only versions whose signature verifies, so a configuration row inserted directly into the database cannot direct a stored secret to a new host. Registering a configuration that would send an existing secret to a host not previously approved requires explicit confirmation, which is recorded in the ledger.

10.3 Token handling and DPoP

OAuth access tokens exist only in broker memory and are refreshed before expiry; they are discarded on an authentication failure, on disabling a connector and on the kill switch. When *dpop: true* is set, the broker generates an ES256 key pair per token, sends a fresh signed proof with every token request and API call (including retries), handles server-supplied nonces, and refuses a token returned as a plain bearer token. A token copied from the hub is therefore rejected by a DPoP-enforcing provider without the corresponding private key. DPoP depends on provider support and cannot protect against an attacker who controls the live broker process.

10.4 IP restriction, capabilities and the ledger

A connector may declare the provider-side IP allow-list, and the hub's egress addresses are configured separately; posture checks report whether they agree. Enforcement of an allow-list is performed by the provider, not by IntegrateX. The capability registry records, for each connector and control, whether the control is enforced, monitored, available, unknown, unsupported or not applicable, so that a public feed without credentials is not reported as missing token binding. Every outbound call, policy denial, secret change and registration is appended to a hash-chained audit ledger in which each entry's hash covers the previous hash and the entry's canonical JSON; the table is append-only through database grants and a trigger.

11. STAGED INGESTION AND INTEGRITY EVALUATION

Writing records straight into shared tables during a run exposes consumers to partial results whenever a run fails part-way. IntegrateX therefore writes each run's records to staging tables keyed by a synchronisation identifier. A synchronisation may span several runs when a run stops early and a later run resumes from its checkpoint. While staging, the runtime collects evidence: the HTTP status and SHA-256 hash of every page, record and quarantine counts, how pagination ended, the newest record timestamp (freshness) and a recursive **schema fingerprint** of the records, which is compared with the last published synchronisation to detect fields that disappeared or changed type.

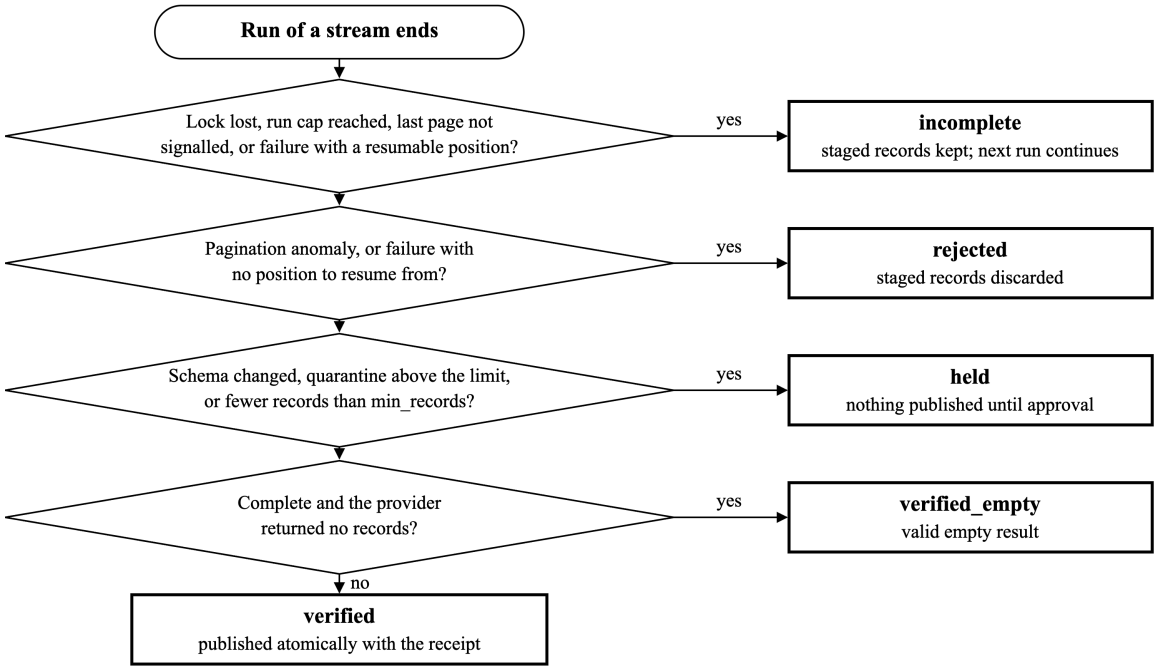


Figure 11.1 Integrity decision flow, in the order evaluated by src/engine/integrity.ts

Figure 11.1 shows how the evidence is converted into one of five decisions. A **verified** synchronisation is complete and within its quality limits (by default no more than 10 % of records quarantined and at least *expect.min_records* accepted); its staged records are published. **Verified_empty** means the provider explicitly returned no records on a complete run. **Held** means the run completed but requires a human decision because of a schema change, excessive quarantine or too few records; nothing is published until an operator approves or discards it, and approval is refused if a newer synchronisation has already been published. **Incomplete** keeps the staged records so that the next run continues the same synchronisation. **Rejected** discards the staged records after a pagination anomaly or a failure with no position to resume from. Publication, the run status, the incremental watermark and the receipt are committed in a single transaction on the database session that holds the run lock, so a watermark can never advance past data that was not published.

12. SYNC INTEGRITY RECEIPTS AND AUDITABILITY

Every run, approval and discard produces a Sync Integrity Receipt stored in the *sync_receipts* table. A receipt is a JSON document whose contents are listed in Table 12.1. Its digest is the SHA-256 hash of a canonical JSON serialisation in which object keys are sorted, so the same content always produces the same digest.

Table 12.1 Contents of a Sync Integrity Receipt

Field group	**Contents**
Identity	Connector, stream, run and synchronisation identifiers; trigger; timestamps
Configuration	Configuration version and its SHA-256 hash
Page evidence	For each page: HTTP status and SHA-256 hash of the response body
Counts	Records seen, accepted and quarantined for the run and for the whole synchronisation
Pagination	How the stream ended, for example the provider's last-page signal or an anomaly
Schema	Schema fingerprint and its difference from the last published synchronisation
Freshness	Newest record timestamp observed
Decision	One of the five decisions, its reasons, the watermark decision and the number of records published
Detections	Foreign-IP detection results for audit-log streams, where configured
Claims	Separate lists of observed facts, inferences, and statements the receipt does not claim

After the transaction that stores a receipt commits, the broker re-reads the receipt, recomputes its digest and appends it once to the hash-chained audit ledger, recording the ledger sequence number in the receipt row. The application database role may insert receipts but cannot update or delete them. Verification with *ix receipt <id>* or the dashboard recomputes the digest from the stored body and compares it with both the stored digest and the ledger anchor; an altered receipt is reported as not matching.

Receipts are integrity evidence, not digital signatures: no public-key signature is applied, and their tamper evidence rests on the ledger chain and database permissions. A receipt documents what IntegrateX observed and decided. It does not prove that the provider's data is factually true or complete, which is why each receipt lists inferences, such as pagination completeness inferred from the provider's last-page signal, separately from observed facts.

13. METHODOLOGY AND PROCESSING WORKFLOW

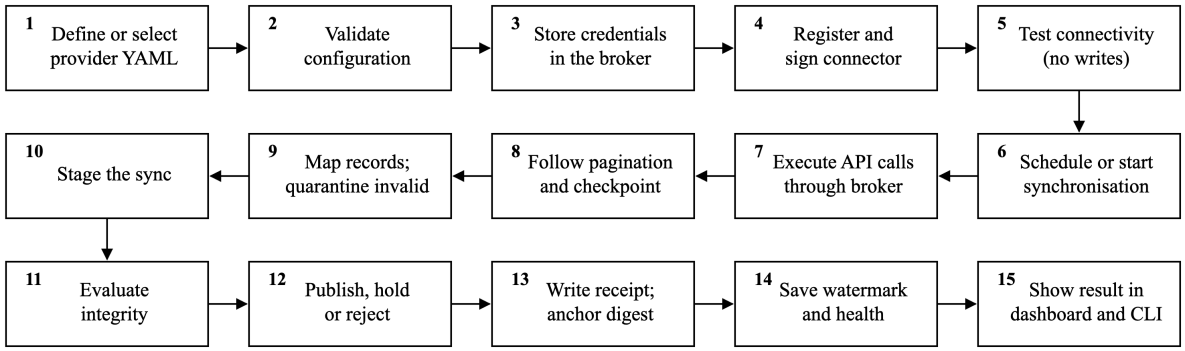


Figure 13.1 End-to-end processing workflow

Figure 13.1 summarises the workflow that every connector follows. The operator first writes or selects a YAML file and validates it with *ix validate*, which reports errors with line numbers. Credentials are stored with *ix secret set*, which reads the value from a hidden prompt and sends it only to the broker. The configuration is then registered with *ix add*, at which point the broker signs it, and *ix test* performs a live authentication and first-page request and shows three mapped records without writing any data. The same steps are available in the dashboard's configuration builder, which generates the YAML from a form.

Synchronisation is triggered by the scheduler, by *ix run* or by the dashboard; all three paths place a job on the same queue. The worker executes API calls through the broker, follows pagination, writes checkpoints, maps records and quarantines those that fail mapping. When the stream ends, the integrity evaluator selects one of the five decisions; the result is published, held or discarded, the receipt is written and anchored, and the watermark and connector health are updated. Operators observe the outcome through the dashboard's health cards, the records, receipts and quarantine views, and CLI commands such as *ix runs* and *ix receipts*. The workflow is deterministic for a given configuration and provider response, which makes it reproducible in tests against mock providers.

14. INCIDENT DETECTION AND RESPONSE

The threat model assumes that an attacker may obtain a copy of the hub's database or credentials, or may control part of the hub. IntegrateX combines prevention-oriented controls (broker isolation, encryption, DPoP and provider-side allow-lists) with detection and containment.

14.1 Detection

Credential canaries are decoy secrets stored and encrypted exactly like real ones; only the broker can recognise them, through an HMAC tag keyed from the master key. If anything inside the hub attempts to use a decoy, the broker refuses the call and raises a critical alert. Decoy webhook URLs point to the hub's own listener, so their use from outside raises an alert containing the caller's IP address regardless of provider. **Foreign-IP detection** applies declarative rules to audit-log streams, such as Okta's System Log: an event that matches the hub's own client identifier but originates from an address outside the declared allow-list raises a *foreign_ip_use* alert and is recorded in the receipt. Detection latency equals the polling interval, so a security alert or the *ix incident start* command activates **incident mode**, in which connectors that declare *incident_every* poll more frequently (for example every 2 minutes instead of 15) for two hours after the last security alert.

14.2 Advisory and response

The **incident advisor** builds an evidence pack from recent alerts, ledger denials and held or rejected receipts, each with a stable identifier. Fixed rules decide the recommendation: a used canary or foreign-IP use is critical and yields *Kill switch required: YES*. Optionally, a large language model (Claude or a Groq-hosted model) writes an explanatory summary; claims that cite identifiers absent from the pack are discarded, a summary that contradicts the rule-based decision is dropped, and alert text is supplied as untrusted data. The advisor never acts on its own.

The **kill switch** is triggered by an authorised operator from the dashboard or with *ix panic*. It disables every connector, cancels queued runs, revokes cached OAuth tokens at providers that expose an RFC 7009 revocation endpoint, discards all in-memory tokens, optionally deletes stored secrets, and returns a per-connector checklist. Static API keys and personal access tokens cannot be revoked by IntegrateX; the checklist marks them for manual revocation at the provider. The application database role cannot re-enable a disabled connector; re-enabling goes through the broker.

15. EVALUATION AND TEST RESULTS

The automated test suite was executed with *pnpm test* on 3 November 2026 for this report. All 230 tests in 18 test files passed. Database-backed tests use a dedicated PostgreSQL test database; provider behaviour is reproduced by a mock server that implements GitHub-, Okta-, NVD- and Jira-style APIs, OAuth with optional DPoP, Okta-style audit logs and injectable faults (HTTP 429, 500 and 503 responses, rate-limit 403 responses, timeouts, dropped connections, malformed or empty bodies and schema drift).

Table 15.1 Test coverage by area (test files in the test directory)

Area	Test files	Type	Tests
Configuration validation and policy	config.validate	Unit	22
Mapping, paths and transforms	mapper	Unit	11
Pagination styles and anomalies	paginate	Unit	12
Retry, backoff and rate limiting	retry, reliability	Unit and mock integration	19
Engine on mock providers; secret-store encryption	engine.e2e, phase4	Mock integration	30
Broker, secrets, signatures, host binding	phase6	Integration (database, sockets)	35
DPoP proof and token binding	dpop	Mock integration	7
Fingerprints, canaries, IP ranges, capabilities, advisor rules	pure	Unit	18
Staging and integrity decisions	staging, integrity	Integration and unit	26
Atomic finalisation and run locks	finalisation, singleflight	Integration (concurrency)	13
Scheduler, worker, API, recovery, incident mode	phase5	Integration	17
Breach features, posture, kill switch rehearsal	phase7, phase8	Integration	19
Full breach-story demonstration	demo-rehearsal	End-to-end on mock provider	1

Per-file counts are taken from the test runner's list of tests and sum to 230; six of the configuration-validation tests are generated, one for each sample configuration file.

Beyond the automated suite, the eight live connectors in Table 7.1 were run against the real provider APIs during development; at the time of writing all eight reported a healthy status and verified receipts, for example 31 repositories from GitHub and 10 detection alerts from Elastic Security. These live runs demonstrate functional correctness for the tested accounts only. No throughput, latency or detection-accuracy measurements were performed, and no external security assessment or certification has been carried out. The mock-provider tests verify behaviour under controlled conditions and should not be read as production-scale validation.

16. APPLICATIONS, CURRENT LIMITATIONS AND FUTURE ENHANCEMENTS

16.1 Applications

IntegrateX is applicable wherever security data from several platforms must be collected into one consistent store: aggregating identity, vulnerability and alert data for security operations; feeding a central repository from vulnerability feeds and detection platforms; supporting audit and compliance work through receipts that show what was collected and how it was judged; and serving as the integration layer of a larger security programme. It is not a replacement for a security information and event management (SIEM) or security orchestration (SOAR) platform; it supplies verified, normalised data that such tools can consume.

16.2 Current limitations

- DPoP, provider IP allow-lists and audit-log detection depend on provider support; GitHub's IP allow list and audit log, for example, require GitHub Enterprise Cloud.
- DPoP and foreign-IP detection were verified end-to-end only against the mock provider, and four vendor configurations have not been exercised against live APIs.
- GraphQL, SOAP, signed-request schemes such as AWS Signature Version 4 and unusual OAuth flows require adapters that do not yet exist.
- In the development setup all processes run as one operating-system user, so code running as that user can read the broker's key files; this weakens the process separation.
- Outbound requests are constrained by host allow-lists and refuse redirects, but private-address filtering and DNS-rebinding protection are not yet implemented.
- Retention periods are declared but not yet enforced, alert rows are not authenticated, and records deleted at the source are not removed.
- Receipts are hash-anchored rather than digitally signed, and an attacker who controls the live broker can use the credentials it holds.

16.3 Future enhancements (planned)

- Outbound network hardening: private and link-local address filtering, connection-time IP checks and DNS-rebinding defence.
- Running the broker under its own operating-system user or in a separate container.
- Enforced data retention, authenticated alert provenance and bounded alert queues.
- Stronger database permissions and tenant isolation.
- Protocol adapters for GraphQL and signed-request APIs, and live validation of additional providers.
- Further response automation, retaining human authorisation for disruptive actions.

17. CONCLUSION AND REFERENCES

17.1 Conclusion

This project addressed two related problems: the cost and inconsistency of building a separate integration for every security platform, and the risk created by an integration hub that holds credentials for all of them. IntegrateX shows that a single provider-neutral engine, driven by validated YAML configurations, can ingest data from APIs with different authentication and pagination schemes into one common store, and that the same engine can apply uniform reliability and security controls to every connector. Its principal contribution is evidence-gated synchronisation, in which data is staged, judged and published atomically, and every decision is recorded in a hash-anchored Sync Integrity Receipt. The credential broker, signed configurations, DPoP where supported, decoy credentials, foreign-IP detection and a human-authorised kill switch reduce what a compromised hub can expose and shorten the time to detect misuse. The implementation is working and tested, with eight connectors verified against live APIs; the limitations in Chapter 16 define the work required before production deployment.

17.2 References

[1] H. Infant, “IntegrateX: configuration-driven security integration platform,” GitHub repository, 2026. [Online]. Available: https://github.com/hasishinfant/mitigata-hack

[2] IntegrateX project documentation: README.md, docs/DETAILS.md, docs/DEMO.md and docs/coverage.md, in [1], 2026.

[3] Mitigata CyberForge, “PRD: Config-driven security integration platform: problem, goals and requirements,” Problem Statement 05, 29 Oct. 2026.

[4] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones and D. Waite, “OAuth 2.0 Demonstrating Proof of Possession (DPoP),” RFC 9449, IETF, Sep. 2023.

[5] T. Lodderstedt, S. Dronia and M. Scurtescu, “OAuth 2.0 Token Revocation,” RFC 7009, IETF, Aug. 2013.

[6] D. Hardt, “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, Oct. 2012.

[7] M. Jones, B. Campbell and C. Mortimore, “JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants,” RFC 7523, IETF, May 2015.

[8] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, IETF, May 2010.

[9] H. Krawczyk, M. Bellare and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, IETF, Feb. 1997.

[10] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST SP 800-38D, Nov. 2007.

[11] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.

[12] M. Nottingham and R. Fielding, “Additional HTTP Status Codes,” RFC 6585, IETF, Apr. 2012.

[13] M. Nottingham, “Web Linking,” RFC 8288, IETF, Oct. 2017.

[14] Official documentation of Node.js, Fastify, PostgreSQL, pg-boss, Zod, undici, jose and Vitest, accessed 2026.

[15] “Salesloft says Drift customer data thefts linked to March GitHub account hack,” TechCrunch, 8 Sep. 2025. [Online]. Available: https://techcrunch.com/2025/09/08/salesloft-says-drift-customer-data-thefts-linked-to-march-github-account-hack/

[16] “News brief: Salesloft Drift breach update and timeline,” TechTarget, 2025. [Online]. Available: https://www.techtarget.com/searchsecurity/news/366630934/News-brief-Salesloft-Drift-breach-update-and-timeline